PYTHON

A Beginner’s Guide to Programming

Learn the fundamentals
Variables, data types, conditions, loops and functions

Build practical skills
Collections, files, errors and object-oriented programming

Start creating
A mini project, practice questions and a learning roadmap

Designed for students and first-time programmers

1. What is Python?

Python is a high-level, general-purpose programming language. It is known for readable syntax and is used in web development, automation, data analysis, artificial intelligence, scientific computing and more.

A Python program is a set of instructions that the computer executes. Python code is commonly saved in files ending with **.py**.

Why learn Python?

- Readable and beginner-friendly syntax.
- A large collection of built-in features and third-party libraries.
- Useful in many fields, including AI, web apps and automation.
- A large community and plenty of learning resources.

Your first program

```
print("Hello, world!")
```

The `print()` function displays information on the screen.

2. Installing and running Python

1. Download Python from **python.org** and install it. On Windows, select the option to add Python to PATH if it is offered.

2. Open Command Prompt or a terminal and check the installation:

```
python --version
```

3. Create a file named `hello.py`, write the code below, save it, and run it from that folder:

```
print("Python is working!")
```

```
python hello.py
```

You can also write and run Python in editors such as Visual Studio Code or IDLE.

3. Variables and data types

A **variable** is a name that refers to a value. Python usually determines the type automatically.

```
name = "Asha"       # string (str)
age = 19            # integer (int)
height = 1.65       # floating-point number (float)
is_student = True   # boolean (bool)

print(name)
print(type(age))
```

Common data types:

Type	Meaning	Example
str	Text	"Python"
int	Whole number	42
float	Decimal number	3.14

Type	Meaning	Example
bool	True or False	True
list	Ordered, changeable collection	[1, 2, 3]
tuple	Ordered, unchangeable collection	(1, 2, 3)
dict	Key-value collection	{"city": "Delhi"}
set	Collection of unique values	{1, 2, 3}

4. Operators

Operators perform calculations and comparisons.

```
a = 10
b = 3

print(a + b)   # addition: 13
print(a - b)   # subtraction: 7
print(a * b)   # multiplication: 30
print(a / b)   # division
print(a // b)  # floor division: 3
print(a % b)   # remainder: 1
print(a ** b)  # power: 1000

print(a > b)   # comparison: True
```

Comparison operators include ==, `!=`, >, <, >= and <=. Logical operators include `and`, `or` and `not`.

5. Getting input and making decisions

Use `input()` to read text typed by a user. Convert it when you need a number.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))

if age >= 18:
    print(name, "is an adult.")
else:
    print(name, "is under 18.")
```

Python uses indentation (usually four spaces) to show which statements belong inside a block. An `elif` branch can check another condition.

6. Loops

Loops repeat instructions. A `for` loop is often used to go through a sequence; a `while` loop repeats while a condition remains true.

```
for number in range(1, 6):
    print(number)

count = 3
while count > 0:
    print("Count:", count)
    count -= 1
```

`range(1, 6)` produces 1, 2, 3, 4 and 5. The ending value is not included. Use `break` to exit a loop and `continue` to skip to its next iteration.

7. Functions

A function is a reusable block of code. Define one with `def`; use `return` to send a result back.

```
def greet(person):
    return "Hello, " + person + "!"

message = greet("Ravi")
print(message)
```

Functions help divide a large program into smaller, easier-to-test pieces.

8. Collections: list, tuple, dictionary and set

Lists are ordered and mutable, so you can add, remove or update items.

```
fruits = ["apple", "banana", "mango"]
fruits.append("orange")
print(fruits[0])     # apple
print(len(fruits))   # number of items
```

A tuple is ordered but cannot be changed after creation:

```
coordinates = (12.9, 77.6)
```

A dictionary stores values using keys:

```
student = {"name": "Asha", "mark": 92}
print(student["name"])
student["mark"] = 95
```

A set stores unique values:

```
unique_numbers = {1, 2, 2, 3}  # becomes {1, 2, 3}
```

9. Strings

Strings are text values. You can access characters, combine strings and use useful methods.

```
word = "python"
print(word.upper())       # PYTHON
print(word.capitalize())  # Python
print(word[0])            # p
print(word[1:4])          # yth

name = "Mira"
print(f"Welcome, {name}!")
```

The `f"..."` syntax is called an f-string and is a convenient way to insert values into text.

10. Files and error handling

Use `with open(...)` to work with files. It closes the file automatically when the block finishes.

```
# Write text to a file
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Learning Python!")

# Read the text back
with open("notes.txt", "r", encoding="utf-8") as file:
    content = file.read()
    print(content)
```

Common file modes include `"r"` (read), `"w"` (write/replace) and `"a"` (append).

Use exception handling to respond to errors instead of letting a program stop unexpectedly.

```
try:
    number = int(input("Enter a number: "))
    print(100 / number)
except ValueError:
    print("Please enter a whole number.")
except ZeroDivisionError:
    print("The number must not be zero.")
```

11. Object-oriented programming (OOP)

OOP organizes code around **objects**. A class defines the data and behavior that its objects can have. `__init__` initializes a new object, and `self` refers to that object.

```
class Student:
    def __init__(self, name, mark):
        self.name = name
        self.mark = mark

    def introduce(self):
        print(f"I am {self.name}. My mark is {self.mark}.")

student1 = Student("Asha", 92)
student1.introduce()
```

OOP concepts include classes, objects, inheritance, encapsulation and polymorphism.

12. Modules, pip and virtual environments

A **module** is reusable Python code that you can import. Python includes many standard-library modules.

```
import math
print(math.sqrt(81))

import random
print(random.randint(1, 10))
```

For third-party packages, use `pip`. A virtual environment keeps one project's packages separate from other projects.

```
# Create a virtual environment
python -m venv .venv

# Windows PowerShell: activate it
.venv\Scripts\Activate.ps1

# Install a package
python -m pip install requests
```

If PowerShell blocks activation, you can use the Command Prompt activation script (`.venv\\Scripts\\activate.bat`) or follow Python's official environment guidance.

13. Mini project: Simple calculator

This small project reads two numbers and an operator, then displays the result.

```
first = float(input("First number: "))
operator = input("Choose +, -, *, or /: ")
second = float(input("Second number: "))
```

```
if operator == "+":
    result = first + second
elif operator == "-":
    result = first - second
elif operator == "*":
    result = first * second
elif operator == "/":
    if second == 0:
        result = "Cannot divide by zero"
    else:
        result = first / second
else:
    result = "Unknown operator"

print("Result:", result)
```

Try improving it: add a loop so the user can perform several calculations, and handle invalid numeric input with `try`/`except`.

14. Practice questions

- 1. What is the difference between an integer and a float?
- 2. What does input() return by default?
- 3. What is printed by range(2, 6)?
- 4. How is a list different from a tuple?
- 5. What is the purpose of a function?
- 6. What does a dictionary store?
- 7. Why is exception handling useful?
- 8. What is the role of __init__ in a class?

Quick answer check

1. Integers are whole numbers; floats can have decimals. 2. A string. 3. 2, 3, 4, 5. 4. Lists are mutable; tuples are immutable. 5. Reuse and organize code. 6. Key-value pairs. 7. It lets a program handle expected errors. 8. It initializes a new object.

15. A simple learning roadmap

- **Step 1:** Practise variables, types, input and output.
- **Step 2:** Solve small problems using conditions and loops.
- **Step 3:** Write functions and work with collections.
- **Step 4:** Build projects that read and write files.
- **Step 5:** Learn OOP, modules, packages and virtual environments.
- **Step 6:** Choose a direction: web development, automation, data science or AI.

Useful official resources

Python downloads and documentation: **https://www.python.org/**
Official tutorial: **https://docs.python.org/3/tutorial/**
Package installer guide: **https://packaging.python.org/**

Keep practising: write code, run it, read the errors, and try again.